

# 微服务拆分设计

---

# 微服务拆分设计

---

## 1. 设计目标

课程要求把后端拆成至少 3 个业务微服务。网关、前端和数据库不计入这 3 个服务。划分按业务能力进行，不按用例数量机械拆分。每个服务单独构建、测试和部署；每张业务表只由一个服务管理，其他服务通过接口或快照使用数据。

详细接口、逐表归属和跨服务失败处理见 微服务接口与数据归属.md。服务划分图为 images/33-MS-SPLIT.svg。

## 2. 改造前架构

当前系统为前后端分离单体后端架构：



后端模块包括：用户与认证、商品与二手商品、订单、评价、店铺、地址、聊天议价、上传。

## 3. 改造后架构



| 服务              | 是否计入 3 个业务服务 | 业务职责                    | 划分理由                         |
|-----------------|--------------|-------------------------|------------------------------|
| user-service    | 是            | 注册登录、资料、密码、地址、角色、信用     | 账号生命周期和收货身份内聚，密码与信用不宜散落到交易服务 |
| product-service | 是            | 商品、二手、店铺、评价、聊天、议价、图片、库存 | 卖家可交易资源集中管理，库存只在商品域扣减和恢复     |
| order-service   | 是            | 下单、支付、取消、发货、收货、订单查询和状态机 | 交易过程与历史快照独立保存，不跨库联表还原当时价格和地址 |
| API Gateway     | 否            | 统一入口、路由、超时、CORS、503 降级  | 基础设施，不负责业务表                  |

三个业务服务分别位于 `services/user-service/`、`services/product-service/`、`services/order-service/`，各自有 `package.json` 和 `Dockerfile`。

## 4. 拆分边界

### 4.1 用户服务

职责：注册、登录、JWT 校验所需的用户快照、资料和密码、收货地址、信用分、卖家角色。

公开接口：

```
POST /api/users/register
POST /api/users/login
GET  /api/users/profile
PUT  /api/users/profile
PUT  /api/users/password
GET  /api/addresses
PUT  /api/addresses
```

内部接口（仅服务间调用）：`GET /internal/users/:id`，`POST /internal/users/:id/credit`，`POST /internal/users/:id/role`。

### 4.2 商品服务

职责：商品与二手商品、店铺认证、评价回复、聊天议价、图片上传、库存预留与释放。

公开接口：

```
GET/POST      /api/products
GET/PUT/DELETE /api/products/:id
GET           /api/products/search
GET           /api/products/recommended
GET           /api/products/mine
GET/POST/PUT/DELETE /api/secondhand/**
```

```
GET/PUT/POST    /api/shops/**
GET/POST/PUT    /api/evaluations/**
GET/POST/PUT    /api/chats/**
POST            /api/uploads/images
```

内部接口：库存预留、释放、完成。

### 4.3 订单服务

职责：创建订单、支付、取消、发货、收货、买家/卖家查询。创建订单时调用商品服务预留库存，不直接读商品表。

公开接口：

```
POST /api/orders
GET  /api/orders
GET  /api/orders/seller
GET  /api/orders/:id
POST /api/orders/:id/pay
POST /api/orders/:id/cancel
POST /api/orders/:id/ship
POST /api/orders/:id/confirm
PUT  /api/orders/:id
```

内部接口：购买证明，供评价和退款校验。

### 4.4 API 网关

职责：统一入口、原路径兼容、JWT 头透传、超时、CORS、依赖失败返回 503。不持有业务表。

## 5. 数据库策略

同一 MySQL 服务器中使用三个独立数据库：softw\_users、softw\_catalog、softw\_orders。每个服务只连接自己的库，不跨库联表。订单中的商品、地址和物流以创建时快照保存。

| 表                                                                                    | 唯一管理服务          |
|--------------------------------------------------------------------------------------|-----------------|
| Users, Addresses                                                                     | user-service    |
| Products, Shops, Evaluations, ChatConversations, ChatMessages, InventoryReservations | product-service |
| Orders                                                                               | order-service   |
| uploads/ 文件                                                                          | product-service |

## 6. 跨服务调用与失败处理

| 调用               | 失败时                          |
|------------------|------------------------------|
| 订单 -> 商品：预留库存    | 不创建订单；超时由网关或调用方返回 503        |
| 订单 -> 商品：释放库存    | 按 reservationId 幂等释放，避免重复加库存 |
| 订单 -> 商品：完成预留    | 失败则订单不进入已完成，允许重试             |
| 商品 -> 订单：评价/退款资格 | 校验失败则不写评价或不发起退款              |
| 商品/订单 -> 用户：身份快照 | 写操作拒绝；公开商品查询仍可用              |
| 商品/订单 -> 用户：信用   | 失败不回滚已完成发货，可重试               |
| 网关 -> 业务服务       | 超时或断开返回 503，不伪造业务结果          |

内部接口需携带 X-Internal-Token。

## 7. 重构步骤

1. 梳理路由、控制器、模型和业务职责。
2. 抽出鉴权、错误处理和内部调用客户端。
3. 网关保持原 /api/\*\* 路径。
4. 用户、商品、订单迁入独立服务和独立数据库。
5. 用库存预留接口替代跨库读写商品表。
6. 对拆分后的核心接口做回归测试。

## 8. 风险控制

| 风险         | 影响       | 控制措施                                 |
|------------|----------|--------------------------------------|
| 拆分过细导致无法演示 | 影响验收     | 只拆 3 个业务服务，网关不计入                     |
| 跨服务调用失败    | 下单或评价中断  | 预留接口幂等；失败不写半成品订单；网关 503              |
| 前端接口变动过大   | 页面不可用    | 网关保持原路径                              |
| 数据不一致      | 库存与订单对不上 | 商品表只由商品服务改库存；订单只保存 reservationId 和快照 |

## 9. 小学期验收对应

- 服务划分图：02\_docs/images/33-MS-SPLIT.svg
- 服务接口清单、数据表归属、跨服务说明：02\_docs/微服务接口与数据归属.md
- 改造前版本：Git 标签 monolith-start
- 改造后代码：services/ 与 03\_devops/docker-compose.microservices.yml